T11 NPC Dialogue

CRE132 worksheet

What you'll make

People to talk to. Walk up to the blacksmith, the villager or the signpost and an **E** appears above them. Press E and a box opens at the bottom of the screen with their name and their first line, typing itself out letter by letter. Press E again to see the whole line, and again for the next one. After the last line the box closes.

You'll write two scripts: **Talkable**, which goes on each person and holds what they say, and **DialogueBox**, one shared box that shows it. You'll also see how to stop one key press doing two jobs at once.



Figure 1: The finished scene: the village square with the blacksmith, the villager and the signpost, and the dialogue box open.

Before you start

1. Unzip `CRE132_T11_NPCDialogue_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T11_NPCDialogue_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T11_NPCDialogue`.
3. Press **Play**. You can walk (WASD or arrows) and swing the sword (J). Every **E** prompt is showing, and so is an empty dialogue box at the bottom: nothing hides them yet.

Under **People** are the **Blacksmith**, the **Villager** and the **Sign**. Each has a **Circle Collider 2D** with **Is Trigger** ticked (its reach) and a child **Prompt** (the "E"). The **Dialogue Canvas** has a **Dialogue Panel** with three texts: **Name**, **Line** and **Hint**.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The shared box: DialogueBox

Talkable needs to call the box, so make the box first. Make a folder `Assets/_game/Scripts/T11_NPCDialogue` and create `DialogueBox` in it:

```
// The box at the bottom of the screen that shows what someone says, one line at a time.
// Letters appear one by one, like a typewriter. Pressing Interact while a line is still
// typing shows the rest of it at once; pressing it after that moves on. After the last
// line the box closes.
using System.Collections;
using TMPro;
using UnityEngine;
using UnityEngine.InputSystem;

public class DialogueBox : MonoBehaviour
{
    [SerializeField] private InputActionReference interactAction;
    [SerializeField] private GameObject panel;
    [SerializeField] private TMP_Text nameText;
    [SerializeField] private TMP_Text lineText;
    [SerializeField] private float lettersPerSecond = 40f;

    // Switched off while talking, so the player stands still and can't swing the sword
    // (TopDownMovement and MeleeAttack).
    [SerializeField] private MonoBehaviour[] disableWhileTalking;

    private string[] lines;
    private int lineIndex;
    private bool isTyping;

    // The same key press opens the box (in Talkable) and moves it on (here). Remembering
    // the frames when it opened and closed stops one press doing both.
    private int openedOnFrame = -1;
    private int closedOnFrame = -1;

    public bool IsOpen
    {
        get { return panel.activeSelf; }
    }

    // Open, or closed this very frame: either way, don't start another conversation yet.
    public bool IsBusy
    {
        get { return IsOpen || Time.frameCount == closedOnFrame; }
    }

    private void Awake()
    {
        panel.SetActive(false);
    }

    private void OnEnable()
    {
        interactAction.action.Enable();
    }
```

```
private void OnDisable()
{
    interactAction.action.Disable();
}

public void Show(string speakerName, string[] newLines)
{
    lines = newLines;
    lineIndex = 0;
    nameText.text = speakerName;
    panel.SetActive(true);
    openedOnFrame = Time.frameCount;

    foreach (MonoBehaviour script in disableWhileTalking)
    {
        script.enabled = false;
    }

    StartCoroutine(TypeLine(lines[lineIndex]));
}

private void Update()
{
    if (!IsOpen || Time.frameCount == openedOnFrame)
    {
        return;
    }

    if (interactAction.action.WasPressedThisFrame())
    {
        if (isTyping)
        {
            ShowWholeLine();
        }
        else
        {
            NextLine();
        }
    }
}

private IEnumerator TypeLine(string line)
{
    // The whole line goes in the text at once, but TextMeshPro only draws the first
    // maxVisibleCharacters letters. Raising it one at a time makes the typewriter.
    lineText.text = line;
    lineText.maxVisibleCharacters = 0;
    isTyping = true;

    while (lineText.maxVisibleCharacters < line.Length)
    {
        lineText.maxVisibleCharacters = lineText.maxVisibleCharacters + 1;
        yield return new WaitForSeconds(1f / lettersPerSecond);
    }

    isTyping = false;
}
```

```
    private void ShowWholeLine()
    {
        StopAllCoroutines();
        lineText.maxVisibleCharacters = lines[lineIndex].Length;
        isTyping = false;
    }

    private void NextLine()
    {
        lineIndex++;
        if (lineIndex < lines.Length)
        {
            StartCoroutine(TypeLine(lines[lineIndex]));
        }
        else
        {
            Close();
        }
    }

    private void Close()
    {
        panel.SetActive(false);
        closedOnFrame = Time.frameCount;

        foreach (MonoBehaviour script in disableWhileTalking)
        {
            script.enabled = true;
        }
    }
}
```

- **`Show`** is the public way in: Talkable hands over a name and an array of lines. `lineIndex` remembers which line we're on.
- **The typewriter** is a coroutine. The whole line goes into the text straight away, but **`maxVisibleCharacters`** tells TextMeshPro to draw only the first few letters. Raising it by one every `1 / lettersPerSecond` seconds makes the letters appear, and the words don't jump about as they type, because the layout is already worked out.
- **E while typing** calls `ShowWholeLine`: `StopAllCoroutines` stops the typing and every letter is shown. **E after that** calls `NextLine`, which types the next line or closes the box after the last one.
- **`disableWhileTalking`** is an array of scripts that are switched off (`enabled = false`) while the box is open. Turning off **TopDownMovement** and **MeleeAttack** keeps the player still.
- **`openedOnFrame` and `closedOnFrame`:** the E press that opens the box reaches this script's `Update` too. Ignoring the frame it opened on stops that same press skipping the first line. `IsBusy` does the same job for Talkable when the box closes (part 2).

Add it: select the **Dialogue Canvas** and add **DialogueBox**. Fill the slots:

- **Interact Action:** click the small circle and pick **Player/Interact** (E, or the gamepad's North button).
- **Panel:** drag in **Dialogue Panel**. **Name Text: Name**. **Line Text: Line**. Leave **Letters Per Second** at 40.
- **Disable While Talking:** click **+** twice. These slots want a *component*, not an object, so right-click the **Player** in the Hierarchy and choose **Properties...**: a small Inspector window opens just for the Player. Drag its **Top Down Movement** component (by its title) into **Element 0**. Close it, do the same with **Player > Sprite** and drag its **Melee Attack** into **Element 1**.

Test it: press Play. The dialogue box is hidden now, but nothing opens it yet.

2. Someone to talk to: Talkable

Create `Talkable` in the same folder:

```
// Something the player can talk to (a villager) or read (a sign). Walk up to it and a
// prompt appears above it; press Interact (E) to show its lines in the DialogueBox.
using UnityEngine;
using UnityEngine.InputSystem;

public class Talkable : MonoBehaviour
{
    [SerializeField] private InputActionReference interactAction;
    [SerializeField] private DialogueBox dialogueBox;
    [SerializeField] private string speakerName = "Villager";
    [SerializeField] [TextArea(2, 4)] private string[] lines;

    // A child object with the "E" prompt, shown while the player is close enough to talk.
    [SerializeField] private GameObject prompt;

    // True while the player is inside this object's trigger (close enough to talk).
    private bool playerInReach;

    private void Awake()
    {
        prompt.SetActive(false);
    }

    private void OnEnable()
    {
        interactAction.action.Enable();
    }

    private void OnDisable()
    {
        interactAction.action.Disable();
    }

    private void Update()
    {
        // The prompt shows when you could start talking: in reach, and not already
        // talking.
        prompt.SetActive(playerInReach && !dialogueBox.IsOpen);

        if (playerInReach && !dialogueBox.IsBusy &&
            interactAction.action.WasPressedThisFrame())
        {
            dialogueBox.Show(speakerName, lines);
        }
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            playerInReach = true;
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
```

```
        if (other.CompareTag("Player"))
        {
            playerInReach = false;
        }
    }
}
```

- **`string[] lines`** is an array of text: one entry per line of speech. **`[TextArea(2, 4)]`** gives each one a box two to four lines tall in the Inspector, so longer lines are easy to type.
- **In reach:** the trigger circle sets `playerInReach` when something tagged **Player** enters, and clears it when it leaves. `Update` shows the prompt only when you're in reach and the box isn't open.
- **`!dialogueBox.IsBusy`:** the E press that closes the box after the last line would also reach this `Update` and start the conversation again straight away. `IsBusy` is still `true` on the frame the box closed, so that press is ignored.
- **The person holds the data, the box does the showing.** A new person is just a Talkable with different lines.

Add it to the **Blacksmith**, the **Villager** and the **Sign**. On each, set **Interact Action** to **Player/Interact**, drag the **Dialogue Canvas** into **Dialogue Box**, and drag that object's own child **Prompt** into **Prompt**. Then type the name and lines (click **+** under **Lines** for each one):

- **Blacksmith**, Speaker Name `Blacksmith`:
  - `Ah, a visitor! Mind the anvil, it bites.`
  - `Slimes have been eating my iron again. Little blue thieves.`
  - `If you find a gold key, the old gate to the east is yours to open.`
- **Villager**, Speaker Name `Villager`:
  - `Hello! Press E next to anyone with a little E above their head to talk.`
  - `Press E again while I'm still talking to see the whole line at once.`
  - `That's it! Go and say hello to the blacksmith.`
- **Sign**, Speaker Name `Sign`, one line:
  - `Mossy Meadow Village. Population: 2 (and a lot of slimes).`

Test it: press Play and walk up to the villager. The **E** appears; press E. The line types itself out; press E halfway to see all of it, then again for the next line. You can't walk or attack while the box is open. After the last line the box closes and the **E** comes back.

Check your work

- ☐ The **E** shows only when you're close to someone, and hides while you're talking.
- ☐ Lines type out letter by letter. E while typing shows the whole line; E again moves on.
- ☐ The box closes after the last line, and the same press doesn't start the conversation again.
- ☐ You can't walk or swing the sword while the box is open, and can again when it closes.
- ☐ The sign works the same way, with one line.
- ☐ Compare with `CRE132_T11_NPCDialogue_Finished` if something behaves differently.

Key points

- **Data on the person, showing in one shared box.** Each Talkable holds who says what; one DialogueBox shows it for everyone.
- **`maxVisibleCharacters`** is the easy, tidy way to make typewriter text in TextMeshPro.
- **When two scripts listen for the same button, decide who gets the press.** Here both scripts remember frame numbers (`Time.frameCount`) so one press does one job.
- **Switching a script off** (`enabled = false`) stops its `Update`, which is a simple way to freeze the player.

If something goes wrong

- **“The type or namespace name ‘DialogueBox’ could not be found”:** DialogueBox has an error, or its file name doesn’t match the class. Fix the first red error in the Console.
- **NullReferenceException in Talkable’s Awake:** its **Prompt** slot is empty.
- **NullReferenceException in Talkable’s Update:** the **Dialogue Box** slot is empty.
- **The E never appears:** the person’s **Circle Collider 2D** isn’t a trigger, or the Player isn’t tagged **Player**.
- **E does nothing:** the **Interact Action** slot is empty on the Talkable or the DialogueBox.
- **The first line is skipped, or the conversation starts again as it ends:** compare the `openedOnFrame` and `IsBusy` lines with the listing.
- **The player can still walk while talking: Disable While Talking** has the wrong components in it (for example **Directional Animator** instead of **Melee Attack**). Use **Properties…** and drag the components themselves.
- **The text appears all at once: Letters Per Second** is very high, or `maxVisibleCharacters` is set after the loop instead of before it.

Try it

1. Change **Letters Per Second** on the DialogueBox to 10, then 100. Why does it change for everyone? How could you give each person their own speed?
2. Add a new person: duplicate the **Villager**, move them, and give them new lines. No code needed.
3. Play a typing sound every few letters, using S05’s **AudioManager**.
4. Give someone different lines the second time you talk to them (an `int timesTalked`).
5. Turn the person to face the player while talking.
6. For conversations with choices, try the optional T11b Yarn Spinner worksheet.